

Whistle Counter — The Full Journey

From First Model to Working App Lab Project

A complete record of what we tried, what failed, what we learned, and where we ended up.

THE GOAL

Build a TinyML-powered pressure cooker whistle counter on the Arduino UNO Q board. The system should listen continuously, detect and count each whistle burst, and display the count on a web interface. Users start and stop counting via a mobile/web app.

The Arduino UNO Q is a dual-processor board — a Qualcomm QRB2210 MPU running Debian Linux and an STM32U585 MCU running Zephyr RTOS. The two communicate via a Bridge RPC layer. All audio processing and inference runs on the Linux MPU side.

CHAPTER 1 — THE FIRST MODEL AND FIRST TEST

Starting Point

The project started with an existing Edge Impulse project (ID 1001902, cooker-whistle-detector) that had a 2-class model — background and cooker_whistle. It was trained on Freesound recordings of pressure cookers in clean studio environments. The model was downloaded as a Linux AARCH64 .eim file and copied to the board.

Setting Up the Test Script

The first challenge was simply getting inference to run. The ImpulseRunner from edge_impulse_linux was used. A series of dependency issues came up one by one:

- numpy not installed in venv
- six not installed — needed by edge_impulse_linux
- opencv missing — warning but not blocking
- Model file needed chmod +x before runner could execute it

Each of these was fixed one step at a time.

First Run — Wrong Classification Mode

The first version of test_whistle.py used classify() with manually pre-windowed data. Every single window returned exactly background=0.79 whistle=0.21 — the same fixed numbers regardless of what was in the audio. This was a sign that the feature input format was completely wrong.

PROBLEM: Model returned fixed output — classify() was receiving wrong format

Discovery — How classify() Actually Works

Inspecting the ImpulseRunner source revealed that classify() takes raw int16 audio samples directly — not normalized floats, not pre-processed MFE features. The model handles MFE preprocessing internally. The correct approach was to pass 16000 raw int16 samples (one full second at 16kHz) as a Python list.

Additionally, the model parameters showed slice_size=4000 and use_continuous_mode=true — meaning the model expected 4000-sample chunks with a sliding window, not manually sliced 16000-sample chunks.

RESOLVED: Rewrote test script to pass raw int16 values, 16000 samples per classify() call with 4000-sample stride

First Successful Detection — WAV File Test

With the corrected script, the Freesound WAV file test produced near-perfect results: whistle=1.00 during actual whistles, background=1.00 during silence, consecutive logic correctly counted whistle events. The model worked excellently on the clean Freesound recordings. This created a false sense of confidence — the hard problem was still ahead.

CHAPTER 2 — THE HISS PROBLEM

Real Kitchen Recording Test

A real-world recording (rec.wav, 70MB, ~7 minutes of actual pressure cooker cooking) was copied to the board and tested. Results were catastrophic:

- Model detected 27, 21, 29, 10 whistles in different sections
- Actual whistle count was 4-5
- Hiss (sustained steam sound) was scoring whistle=1.00 continuously
- Cooldown of 2.5 seconds meant hiss re-triggered a new count every 2.5 seconds

PROBLEM: 2-class model cannot distinguish pressure hiss from whistle — both score 1.00 whistle confidence

Root Cause Analysis

The pressure cooker whistle and the sustained steam hiss that precedes and follows it have overlapping frequency content. Both are tonal sounds in the 1.5-3kHz range. The 2-class model (background vs whistle) has never seen hiss as a separate category so it classifies it as whistle — the closest match.

Key insight: A 2-class model cannot distinguish acoustically similar sounds that fall between classes. Hiss is neither background nor a classic cooker whistle burst — it needs its own class.

The Decision — Add Hiss as Third Class

The decision was made to retrain with 3 classes: background, cooker_whistle, and pressure_hiss. This was the architecturally correct solution. The alternative — tuning threshold and cooldown — could reduce false positives but could never reliably eliminate them when hiss confidence scores as high as 1.00.

RESOLVED: Committed to retraining with 3-class model in a new Edge Impulse project (ID 1016427, whistle-detector)

CHAPTER 3 — THE MICROPHONE PROBLEM

Original Plan — INMP441 I2S Microphone

The original design assumed an INMP441 digital I2S microphone would be wired to the board's MCU headers. The INMP441 was already wired to SPI2 pins (PB13/PB14/PB15). The assumption was that these could be used for I2S communication.

PROBLEM: SPI pins cannot do I2S — these are completely different protocols

Investigation 1 — Zephyr Device Tree

The board's Zephyr device tree was inspected to find what audio peripherals actually exist. The grep command revealed SPI1/SPI2/SPI3 available on headers, SAI1_A and SAI1_B dedicated audio peripherals exist in the chip, but no I2S node. SAI (Serial Audio Interface) is exactly what the INMP441 needs — it is STM32's dedicated audio peripheral that supports I2S natively. But both SAI nodes had status=disabled and no pinctrl assigned. Arduino's Zephyr configuration had deliberately disabled SAI.

PROBLEM: SAI exists in hardware but is disabled in Zephyr DTS — cannot be used via App Lab and it may lose data

Investigation 2 — MPU Side (QRB2210 Linux)

The datasheet revealed the QRB2210 MPU has MI2S0 — a dedicated I2S audio bus. This could theoretically connect to the INMP441. But the datasheet explicitly states that MI2S0 lines are interface-dedicated, operate at 1.8V, and are reserved in the Linux kernel device tree. INMP441 operates at 3.3V — voltage mismatch, and MI2S0 is not accessible to user apps.

PROBLEM: MPU I2S path blocked by voltage mismatch and kernel reservation

Investigation 3 — App Lab Library Import

The Edge Impulse Zephyr library was considered as a way to run inference on the MCU with audio from SPI bit-bang I2S. But App Lab's sketch library manager only shows community Arduino libraries — there is no way to import a custom ZIP library.

PROBLEM: Cannot import custom libraries in App Lab — Edge Impulse Zephyr lib blocked and the ram size isn't compatible

The WSL Mic Adventure

An attempt was made to test with a laptop microphone via WSL. This turned into a long debugging journey:

- WSL1/WSL2 does not pass through audio hardware by default — 0 devices found
- Downloaded PulseAudio for Windows — wrong architecture binary
- Found correct version — config file path was different from expected
- PulseAudio started but crashed on Unix socket module
- Disabled Unix socket module — still not listening on TCP port 4713
- Discovered WSLg 1.0.73 has built-in PulseAudio via PULSE_SERVER env var
- Installed pulseaudio-utils and libasound2-plugins
- Created ~/.asoundrc to route ALSA through PulseAudio
- Finally got 2 devices visible — pulse and default

But even with mic working in WSL, the .eim binary is compiled for Linux AARCH64 — it cannot run on x86_64 WSL. A separate x86_64 .eim was downloaded but it is still a Linux ELF binary — it cannot run on Windows even via PowerShell.

PROBLEM: .eim is Linux-only binary — cannot run on Windows at all, even with correct architecture

Resolution — USB Microphone

After exhausting the INMP441 and WSL paths, the practical solution emerged: a USB microphone plugged into the USB-C hub that was already connected to the board for the display. Linux sees it immediately as an ALSA device (hw:1,0, USB Audio Device). No drivers, no configuration, no kernel work needed.

The arecord test confirmed it worked at 44100Hz natively. The model expects 16000Hz so the Python script resamples using numpy.interp — a simple linear resampler that works well for audio downsampling.

RESOLVED: USB mic via existing hub — hw:1,0 — works perfectly with no setup

CHAPTER 4 — APP LAB INTEGRATION

Moving to App Lab

The project was restructured as a proper App Lab project at ~/ArduinoApps/whistle/. App Lab provides a managed runtime with bricks — pre-built components for common functionality. The AudioClassification brick handles the USB mic and model inference automatically.

The AudioClassification Brick Discovery

The brick has a clean API but with important constraints discovered through trial and error:

- on_detect callback takes zero arguments — the brick API doc is explicit about this
- The brick uses its own internal model reference (app.yaml brick config) — not the .eim file in models/
- No raw confidence scores are exposed — brick fires on_detect or nothing
- classifier.start() must be called for the brick to begin listening

Important: The brick is a black box. If you need raw confidence scores (bg=0.xx, whistle=0.xx per window) you must use ImpulseRunner directly in test_whistle.py — the brick does not expose this.

WebUI socket.io Version Problem

The WebUI brick serves files from the assets/ folder. A socket.io file was needed for the browser interface. Three different approaches were tried:

- CDN link (/socket.io/socket.io.js) — served by brick server but returned 'unsupported version' error
- Local socket.io.min.js at /assets/libs/ path — version mismatch with brick server
- Local file at /libs/socket.io.min.js (assets/ prefix stripped) — WORKED

The key discovery was that the WebUI brick strips the assets/ prefix when serving files. So assets/libs/socket.io.min.js is served at /libs/socket.io.min.js.

RESOLVED: Copy socket.io.min.js to assets/libs/ and reference it as /libs/socket.io.min.js

WebUI on_message Callback Signature

The WebUI brick passes two arguments to message callbacks: sid (socket ID) and data. The initial callbacks were written with one optional argument (def on_start(_=None)) which caused errors: 'on_start() takes from 0 to 1 positional arguments but 2 were given'.

RESOLVED: Changed all callbacks to def on_start(sid, data=None)

Consecutive Count Bug

A subtle bug was identified in the counting logic: consecutive_count was never reset when the brick stopped firing between whistle bursts. If the brick fired 2 times, then stopped for a while, then fired once more — consecutive_count was already at 2, so the single new detection immediately confirmed a whistle.

RESOLVED: Added CONSECUTIVE_GAP = 2.0 — reset consecutive_count if more than 2 seconds pass between detections

CHAPTER 5 — RETRAINING WITH REAL KITCHEN DATA

Why Retraining Was Unavoidable

Even with the 3-class model structure, the model was trained on phone recordings. Phone recorder apps apply Automatic Gain Control (AGC), noise suppression, and echo cancellation. The resulting recordings

sounded clean — fan noise was suppressed, ambient sounds were reduced. On the board, the USB mic captures raw audio with no processing. Fan noise comes through at full volume. The model had never heard fan noise at this level — it classified it as hiss or whistle instead of background.

Root cause: Train/deploy mismatch caused by phone app audio preprocessing. The model learned clean audio but the deployment microphone captures raw audio.

New Edge Impulse Project — 3 Classes

A new project (ID 1016427, whistle-detector) was created with 3 classes: background (fan, exhaust, kitchen ambient, people talking, sizzling, utensils, silence), pressure_hiss (sustained steam hiss before and after whistles), and cooker_whistle (actual sharp whistle bursts). The dataset had approximately 14 minutes per class, balanced and representative. Data was collected using a phone — which, as we would discover, was itself a problem.

MFCC vs MFE Feature Comparison

The first training run used MFCC (Mel Frequency Cepstral Coefficients). The feature explorer showed all three classes in one big mixed blob — no separation at all. This meant the model would have almost no signal to learn from.

Switching to MFE (Mel Filterbank Energy) immediately improved the feature explorer — classes began to form distinct clusters, especially the whistle class which started separating to a different region. MFE captures raw energy in mel filterbanks without the additional cepstral step — this turns out to be better for tonal non-voice sounds like whistles.

RESOLVED: Switched to MFE — better feature separation confirmed in feature explorer

EON Tuner Failures

Multiple attempts to run the EON Tuner failed with all 6 model runs showing errors (100ms latency, full RAM/ROM usage — clearly error values). The cause was having both MFCC and MFE processing blocks present simultaneously in the impulse. The EON Tuner cannot handle multiple DSP blocks and fails silently.

PROBLEM: EON Tuner fails when both MFCC and MFE blocks are present — must remove unused block first

Manual Training Results

Training manually with Dense 20 + Dense 10 architecture, 100 cycles, learning rate 0.001 produced: overall accuracy 91.9%, cooker_whistle F1: 0.95, pressure_hiss F1: 0.89, background F1: 0.92. This was a significant improvement over the 2-class model which could not separate hiss from whistle at all.

Real Kitchen Test — Fan Noise Problem

The new 3-class model was deployed and tested with the USB mic in a real kitchen environment. Fan noise was now classified as hiss instead of whistle — a major improvement. But hiss still occasionally triggered brief whistle spikes at burst transitions.

The high-pass filter approach (scipy Butterworth 1000Hz cutoff) was applied in test_whistle.py to reduce low-frequency fan content before feeding to the model. This further reduced false activations.

PROBLEM: Still some false positives — root cause is phone-recorded training data not matching raw board mic audio

The proper fix: Record all training data using the same hardware chain as deployment — lavalier mic + Arpana USB dongle — with no AGC or processing applied.

CHAPTER 5b — THE DATA COLLECTION HARDWARE JOURNEY

Why the Phone Recording Failed

The initial training data was recorded on a phone. Even with apps claiming to record in 'raw' mode, Android phones apply noise suppression, AGC, and echo cancellation at the HAL (Hardware Abstraction Layer) — at driver level, below the app. This cannot be disabled by the recording app. The resulting audio had fan noise suppressed, hiss levels altered, and amplitude normalised — a completely different signal from what the board's USB mic captures in raw mode.

Key insight: The phone AGC mismatch is not fixable by tuning thresholds or cooldown timers. It requires discarding all phone recordings and collecting fresh data with the correct hardware.

Phone Playback Re-Recording Mistake

Audio recorded correctly on the board mic was played back through a phone speaker and re-recorded in an attempt to create training data. This was a fundamental error. The phone speaker has limited frequency response (rolls off below ~200Hz and above ~8kHz), adds room reverb and echo, and compresses loud transients. The spectrogram of playback-recorded audio looks completely different from direct capture.

PROBLEM: Playback re-recording produces a speaker frequency response artifact — the model learns the phone speaker, not the cooker

RESOLVED: Discard all playback re-recordings. Record directly from the sound source using the correct hardware chain.

The Correct Hardware Chain

The deployment hardware chain on the board is: lavalier mic capsule → 3.5mm TRRS cable → Arpana USB audio dongle (ADC) → USB hub → board. The Arpana dongle is a generic USB audio device with no onboard DSP. Training data must go through this exact same chain.

Recording on the Board Directly — arecord

The board runs Debian Linux and arecord is available. Recording directly on the board with the USB mic is the perfect solution — zero difference from deployment.

```
arecord -D hw:1,0 -f S16_LE -r 44100 -c 1 -d 120 session.wav
```

The mic native rate is 44100Hz. Recording at 16000Hz via arecord fails because the hardware does not support it. Always record at 44100Hz and let Python resample to 16000Hz using numpy.interp.

Laptop + Same Dongle + Audacity — Valid Alternative

Carrying the board to the kitchen for recording adds friction. The alternative — laptop + same Arpana dongle + Audacity WASAPI exclusive mode — is equally valid. Since the Arpana dongle is the ADC in the chain, and the same dongle is used on both the laptop and the board, the signal path is identical.

Windows Setup for Raw Recording

- Audacity v3.7.7 (64-bit, direct GitHub installer — not via MuseHub)
- Audio Setup → Audio Settings → Host: Windows WASAPI
- Recording Device: Microphone (USB Audio Device) — the Arpana dongle
- Channels: 1 (Mono), Project Sample Rate: 44100 Hz
- Control Panel → Sound → Recording → USB mic Properties → Advanced tab → Default Format: 1 channel, 16 bit, 44100 Hz — both Exclusive Mode checkboxes ticked
- No Enhancements tab on this driver = no driver-level processing — confirmed clean
- Export: File → Export Audio → WAV (Microsoft) → Signed 16-bit PCM

Why WASAPI exclusive matters: In shared mode, Windows may apply noise suppression and AGC. Exclusive mode bypasses the OS audio stack entirely — Audacity talks directly to the USB driver.

Consequences of Changing the Dongle

Different USB audio dongles have different ADC chips with different frequency response, input impedance, preamp gain, and noise floor. For this project specifically — whistle and hiss are strong and spectrally distinct signals — minor ADC differences between identical dongle models will not collapse the model. However, switching to a completely different dongle class (e.g. a high-gain broadcast interface vs a generic ₹300 dongle) could shift amplitude levels enough to affect confidence scores.

Rule: Same Arpana dongle on both laptop and board = safe. Different brand dongle = verify with spectrogram comparison before using for training.

Why Cheap Voice Recorders Don't Work

Several low-cost voice recorders were evaluated as alternatives (YIMBLI, FREDI HD Plus, ISD1820 module). All failed the requirements:

- YIMBLI and FREDI: default to MP3 output (lossy — destroys frequency content), apply internal noise reduction, maximum sample rate often 8000Hz or 16000Hz
- ISD1820: a hobbyist analog playback IC — records to chip, no file output, no WAV, completely wrong product

Requirements any recorder must meet to be usable: WAV PCM output (not MP3), sample rate 44100Hz or 48000Hz, no AGC or noise reduction (or explicit disable), 3.5mm external mic input or acceptable inbuilt mic, SD card or USB file transfer.

Zoom H1essential and Tascam DR-05X meet all requirements but cost Rs. 8000-10000 and use a different mic from the board deployment mic. For this project, laptop + Arpana dongle + Audacity is the better solution at zero additional cost.

Field Recorder Evaluation

A field recorder is a battery-powered handheld device that records directly to SD card as WAV PCM with no computer needed. The idea was evaluated as an option for kitchen recording without carrying the board or laptop. Two sub-questions were explored: (1) use a field recorder with its own inbuilt mic, and (2) connect our own lavalier + Arpana dongle to the recorder to keep the same ADC in the chain.

Requirements for Any Recorder to Be Usable

- Output format: WAV PCM only — MP3 is lossy, destroys frequency content, unusable
- Sample rate: 44100Hz or 48000Hz minimum — 8000Hz/16000Hz recorders miss high-frequency whistle content
- No AGC or noise reduction — must be disableable or absent entirely
- 3.5mm external mic input (if inbuilt mic is not acceptable) or SD card / USB file transfer

Devices Evaluated

- YIMBLI digital recorder — FAIL. Outputs MP3, applies internal noise reduction, sample rate capped at 16000Hz. Designed for meeting transcription, not raw audio capture.
- FREDI HD Plus — FAIL. Same class as YIMBLI. MP3 output, internal noise reduction, low sample rate. No way to disable processing.
- ISD1820 module — FAIL. This is a hobbyist analog voice playback IC used in embedded projects. It records to an onboard chip, not a file. No WAV output, no SD card, no USB. Completely wrong product category.

- Zoom H1essential (Rs. 8000-10000) — PASS requirements. Truly raw WAV PCM, 44100Hz, no onboard processing, 3.5mm mic input, SD card. But uses a different mic capsule from the board deployment mic — introduces mic-to-mic variation.
- Tascam DR-05X (Rs. 8000-10000) — PASS requirements. Same verdict as Zoom H1essential. Reliable, raw, but expensive and different mic from deployment.

The “Connect Our Own Mic” Idea

The idea was: plug the lavalier mic’s 3.5mm jack into a field recorder’s external mic input, so the same mic capsule is used. This would keep the microphone consistent with deployment. However, this doesn’t actually solve the core problem — the Arpana dongle is the ADC in the deployment chain, not just the mic capsule. Plugging the 3.5mm into a field recorder means the field recorder’s internal ADC is doing the conversion, not the Arpana dongle. Different ADC = different frequency response = different signal.

Conclusion: For this specific project — whistle and hiss are strong, spectrally distinct signals — mic-to-mic and ADC-to-ADC variation is unlikely to collapse the model. But the field recorder approach costs Rs. 8000+ and still requires SCP-ing files to a laptop for segmentation. The laptop + Arpana dongle + Audacity solution costs nothing extra, keeps the exact same ADC in the chain, and produces files directly on the machine used for segmentation. Field recorder path abandoned in favour of laptop approach.

CHAPTER 6 — WHAT WE BUILT AND WHERE IT STANDS

Current Working System

- USB mic (lavalier + Arpana dongle) connected to board via USB-C hub — detected as hw:1,0
- App Lab project running AudioClassification brick and WebUI brick
- WebUI accessible at <http://192.168.1.156:7000>
- Start/Stop/Reset buttons working via socket.io
- Consecutive detection logic with CONSECUTIVE_GAP = 2.0s preventing stale counts
- Detection pulse feed showing brick firing events in real time
- test_whistle.py for manual testing with full confidence score output
- High-pass filter at 1000Hz applied before inference in test script

The Remaining Gap

The model performs well on test recordings but has false positives in live kitchen conditions due to the train/deploy mismatch from phone-recorded training data. The fix is clear — retrain using the correct hardware chain (laptop + Arpana dongle + Audacity WASAPI, or board + arecord directly) with real kitchen recordings including fan running.

Architecture Summary

- USB mic (hw:1,0) → AudioClassification brick → on_detect callback → consecutive/cooldown logic → WebUI
- test_whistle.py: USB mic (hw:1,0) → pyaudio → resample 44100→16000Hz → highpass filter → ImpulseRunner → confidence scores
- MCU sketch: just Bridge.begin() — no audio involvement

CHAPTER 7 — COMPLETE PROBLEM LIST

#	Symptom	Cause	Fix	Status
1	classify() returned fixed background=0.79 whistle=0.21 for every window	Passing normalized float32 values instead of raw int16	Pass raw int16 samples as Python list	Resolved
2	Hiss classified as whistle — counts of 27, 21, 29 instead of actual 4-5	2-class model has no hiss class — hiss scores as whistle	Add pressure_hiss as third class and retrain	Resolved
3	INMP441 produced no audio despite correct wiring to SPI2	SPI pins cannot do I2S. SAI disabled in Zephyr DTS. MPU MI2S0 is 1.8V and kernel-reserved.	Use USB microphone via USB-C hub	Resolved
4	Model file is not executable error	Downloaded .eim files do not have execute permission	chmod +x modelfile.eim	Resolved
5	pyaudio sees 0 devices in WSL	WSL does not pass through audio hardware by default	Use board USB mic instead — WSL path abandoned	Resolved
6	.eim cannot run on Windows — WinError 193	.eim is a Linux ELF binary — cannot run on Windows	Run only on Linux (the board)	Resolved
7	Socket.IO unsupported version error in WebUI	CDN URL not served correctly. Local file version mismatch.	Copy socket.io.min.js to assets/libs/ — reference as /libs/socket.io.min.js	Resolved
8	on_start() takes from 0 to 1 positional arguments but 2 were given	WebUI brick passes (sid, data) to all callbacks	All callbacks: def callback_name(sid, data=None)	Resolved
9	ModuleNotFoundError: No module named audio_classifier	Module is audio_classification not audio_classifier	from arduino.app_bricks.audio_classification import AudioClassification	Resolved
10	Invalid sample rate error when recording at 16000Hz	USB mic hardware only supports 44100Hz natively	Record at 44100Hz, resample to 16000Hz using numpy.interp	Resolved
11	All 3 classes in one blob in feature explorer	MFCC not optimal for tonal non-voice sounds	Switch DSP block to MFE	Resolved
12	EON Tuner all 6 runs showing errors	Both MFCC and MFE blocks present simultaneously	Delete unused DSP block before running EON Tuner	Resolved

#	Symptom	Cause	Fix	Status
13	Single detection immediately confirming whistle after partial build-up	consecutive_count not reset between separate whistle events	Reset if more than CONSECUTIVE_GAP (2.0s) passes between on_detect calls	Resolved
14	Phone-recorded training data causes false positives with board USB mic	Phone AGC/noise suppression at HAL level — train/deploy mismatch	Record training data with laptop + Arpana dongle + Audacity WASAPI exclusive, or board + arecord	Pending — requires re-recording
15	Playback re-recording via phone speaker ruins audio	Phone speaker frequency response + reverb + compression changes spectrogram completely	Discard all playback recordings. Record directly from source.	Resolved — data discarded

CHAPTER 8 — WHAT THE NEXT SESSION NEEDS TO DO

Immediate Priority — Fix the Training Data

Everything else is working. The only remaining problem is the train/deploy mismatch. Record new training data using one of the two valid methods:

Method A — Laptop + Arpana dongle + Audacity (recommended, easier):

- Plug lavalier mic + Arpana dongle into laptop
- Audacity: WASAPI exclusive, Microphone (USB Audio Device), 1 (Mono), 44100 Hz
- Record one continuous session: fan on, full cooker cycle start to finish
- Export as WAV, Signed 16-bit PCM
- Segment into clips in Audacity using spectrogram view
- Upload to Edge Impulse project 1016427 (whistle-detector)

Method B — Board + arecord directly:

```
arecord -D hw:1,0 -f S16_LE -r 44100 -c 1 -d 600 full_session.wav
```

- Carry board + hub + mic to kitchen
- SSH in and run arecord command above
- SCP the file to laptop for segmentation

After New Data Is Collected

- Retrain with MFE + Dense 20+10, 100 cycles, learning rate 0.001
- Remove any unused DSP block before running EON Tuner
- Deploy Arduino UNO Q build → download .eim → copy to ~/ArduinoApps/whistle/models/
- Update model reference in app.yaml brick config
- Test with test_whistle.py before deploying via App Lab

Key Facts for Reference

Item	Value
Board	AQ1, IP 192.168.1.156, SSH arduino@192.168.1.156
Venv	~/ArduinoApps/app_to_lcd/ble_display/venv/ (shared with BLE Hub)
Project path	~/ArduinoApps/whistle/
Active model	~/ArduinoApps/whistle/models/whistle-v1.eim (3-class, 91.9% accuracy)
Test script	~/ArduinoApps/whistle/test_whistle.py (ImpulseRunner + high-pass filter)
WebUI	http://192.168.1.156:7000
Edge Impulse project	ID 1016427, whistle-detector
USB mic device	hw:1,0 — USB Audio Device (Arpana dongle + lavalier mic)
Recording format	44100Hz, 16-bit PCM, mono — resample to 16000Hz in Python
Training software	Audacity v3.7.7, WASAPI exclusive, 44100Hz mono 16-bit PCM

Critical Rules — Do Not Violate

- Never record training data on a phone — AGC at driver level ruins the signal
- Never record playback through a phone speaker and re-record
- Never use the laptop built-in mic — Intel Smart Sound processing cannot be disabled
- Always use WASAPI exclusive mode on Windows — shared mode may apply enhancements
- Always record at 44100Hz (mic native rate) — never force 16000Hz at record level
- Never run EON Tuner with both MFCC and MFE blocks present simultaneously
- socket.io must be at assets/libs/socket.io.min.js — referenced as /libs/socket.io.min.js
- Always discuss plan before writing any code — never give code during discussion